

Module 04

LLM Fundamentals

Bangun → Pakai → Produksikan → Adapt → Ukur



5 notebook · gratis di Colab Tesla T4 16GB · Navasena AI · 2026

Act 1

Apa & Mengapa LLM

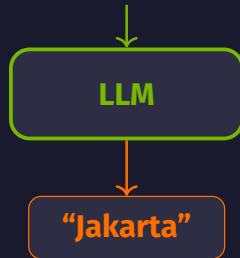
Kenapa model bahasa raksasa mengubah segalanya

Apa itu LLM?

Mesin menebak kata berikutnya yang sangat pintar

- ▶ **LLM** = *Large Language Model*: model bahasa raksasa
- ▶ Belajar dengan membaca teks sangat banyak (miliaran kata dari internet)
- ▶ Tugas intinya satu & sederhana: **menebak kata berikutnya**
- ✓ Ulangi tebakan itu terus-menerus → lahir kalimat utuh
- ▶ Dibangun di atas arsitektur **transformer** (sejak 2017)

“Ibu kota Indonesia adalah ...”

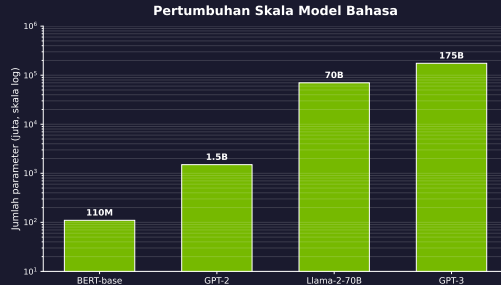


Data sangat banyak + **transformer** = tebakan kata berikutnya yang sangat jitu. Itulah “kepintaran” LLM.

Dari Membaca Satu-satu ke Membaca Sekaligus

Evolusi singkat menuju transformer

- ▶ Dulu: model membaca kata **satu per satu** (RNN/LSTM) — lambat & mudah lupa
- ▶ **Transformer** (2017): membaca **seluruh kalimat sekaligus** lewat **attention**
- ▶ Makin banyak **parameter** (“kenop”) → makin banyak pola yang bisa ditangkap
- ▶ Tapi kualitas **data** sama pentingnya dengan ukuran



Transformer memicu ledakan ukuran: dari ratusan juta jadi ratusan miliar parameter.

LLM di Kehidupan Sehari-hari

Satu model, banyak tugas bahasa

- ✓ **Chatbot & asisten:** ChatGPT, Gemini, layanan pelanggan
- ✓ **Bantu menulis kode:** membuat & menjelaskan program
- ✓ **Terjemahan:** dari satu bahasa ke bahasa lain
- ✓ **Ringkasan:** memadatkan dokumen panjang
- ✓ **Tanya-jawab:** menjawab pertanyaan informatif

Di modul ini kita akan **bangun, pakai, produksikan, adapt,** dan **ukur** LLM — semuanya gratis di Colab T4.

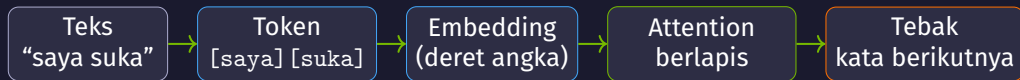
Act 2

Bagaimana LLM Bekerja

Dari teks sampai kata berikutnya

Alur Besar: Bagaimana Transformer Bekerja

Teks → token → embedding → attention berlapis → kata berikutnya



Analogi: bayangkan membaca kalimat yang kata terakhirnya ditutup, lalu kamu menebaknya. Otakmu memakai *semua* kata sebelumnya. Transformer melakukan hal yang sama — tapi dengan angka dan secara paralel. Tiap kotak di atas kita bedah di slide berikutnya.

Langkah 1 — Memecah Teks jadi Token

Komputer tidak membaca huruf; ia membaca potongan ber-nomor

- ▶ Teks dipecah jadi **token** — biasanya potongan kata
- ▶ Tiap token diberi sebuah **nomor** (ID)
- ▶ Kata umum = 1 token; kata jarang dipecah jadi beberapa
- ▶ Di notebook 00 kita pakai cara paling sederhana: **1 huruf = 1 token**



(nomor hanya ilustrasi)

Semua teks diubah jadi **token ber-nomor** dulu sebelum diolah model.

Langkah 2 — Embedding: Mengubah Makna jadi Angka

Inilah cara LLM “memahami” bahasa

- ▶ Tiap token diubah jadi **deret angka** (disebut *vektor*)
- ▶ Aturannya: **makna mirip** → **angka mirip**;
makna jauh → angka jauh
- ▶ “raja” dekat “ratu”; “kucing” jauh dari
“traktor”
- ▶ Jadi “memahami” di sini = menaruh tiap kata
di *peta angka* sesuai maknanya
- ▶ Model menyusun peta ini sendiri dari **pola**
dalam teks yang sangat banyak



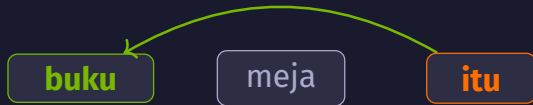
LLM “memahami” = mengenali **pola** lalu menaruh makna sebagai **angka**. Ini statistik dan pola — bukan kesadaran atau perasaan seperti manusia.

Langkah 3 — Attention: Memakai Konteks

Model belajar kata mana yang saling berhubungan

- ▶ **Attention** = model “memperhatikan” kata yang paling relevan
- ▶ Saat mengolah satu kata, ia menimbang semua kata lain di kalimat
- ▶ Contoh: model belajar “**itu**” merujuk ke “**buku**”, bukan “meja”
- ▶ Inilah cara LLM memakai **konteks** — kunci memahami kalimat panjang

“...menaruh buku di meja karena itu berat”



attention: “itu” → “buku”

Attention membuat model fokus pada kata yang benar-benar relevan, bukan menganggap semua kata sama penting.

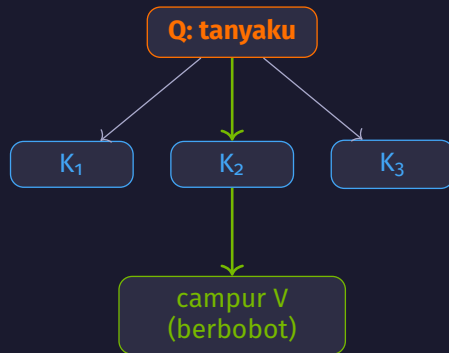
Q, K, V — Analogi Diskusi Kelompok

Mesin di dalam attention, tanpa rumus

Bayangkan satu kelas sedang diskusi. **Tiap kata adalah seorang murid:**

- ▶ **Query (Q)** = *pertanyaanku*: “murid mana yang relevan untukku?”
- ▶ **Key (K)** = *name-tag keahlian* tiap murid: “aku tahu soal X”
- ▶ **Value (V)** = *isi informasi* yang dibawa tiap murid

Cara kerjanya: cocokkan **Q**-ku ke semua **K**. Murid yang paling cocok dapat *bobot* besar, lalu aku ambil *campuran-berbobot* dari **V** mereka.



garis tebal = bobot besar (paling cocok)

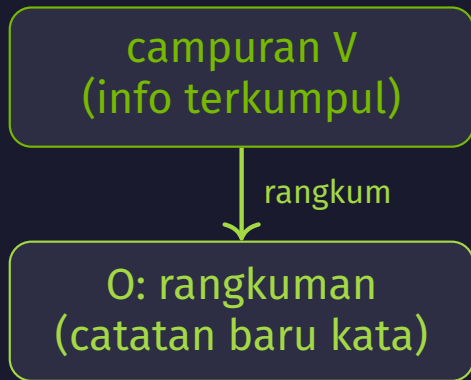
$Q = \text{pertanyaan}$ · $K = \text{label keahlian}$ · $V = \text{isi info}$. Attention = ambil campuran V yang paling cocok.

O — Merangkum Hasil Diskusi

Output projection: langkah terakhir attention

Setelah tiap kata mengambil campuran info dari kata lain, hasilnya masih “mentah”. Perlu satu langkah terakhir:

- ▶ **Output projection (O)** = merangkum info yang terkumpul jadi **catatan baru** untuk kata itu
- ▶ Analogi: setelah bertanya ke teman-teman sekelas, kamu **menulis ringkasan** di catatanmu sendiri
- ▶ Catatan baru inilah yang dipakai lapisan berikutnya

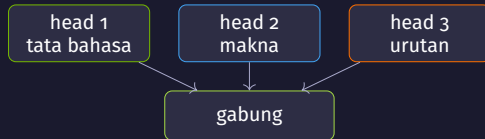


Urutan lengkap: **Q** tanya → cocokkan ke **K** → ambil campuran **V** → **O** merangkum jadi catatan baru.

Langkah 4a — Multi-Head: Banyak Diskusi Paralel

Beberapa attention sekaligus, lalu digabung

- ▶ **Multi-head** = jalankan beberapa attention sekaligus (tiap satu disebut *head*)
- ▶ Tiap head fokus pada **hubungan berbeda**: ada yang lihat tata bahasa, ada yang lihat makna, ada yang lihat urutan
- ▶ Analogi: **beberapa kelompok diskusi** berjalan paralel, lalu semua hasilnya digabung jadi satu



Banyak sudut pandang sekaligus → model menangkap lebih banyak jenis hubungan antar-kata.

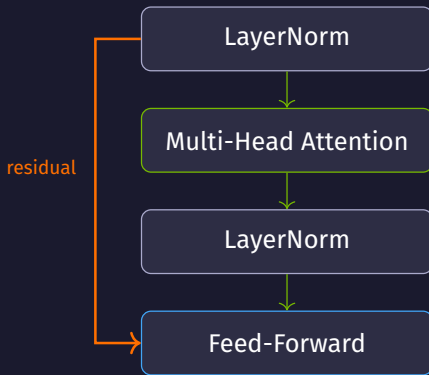
Langkah 4b — Satu Lapisan Transformer (Block)

Empat bagian yang ditumpuk berulang-ulang

Satu **transformer block** berisi empat bagian:

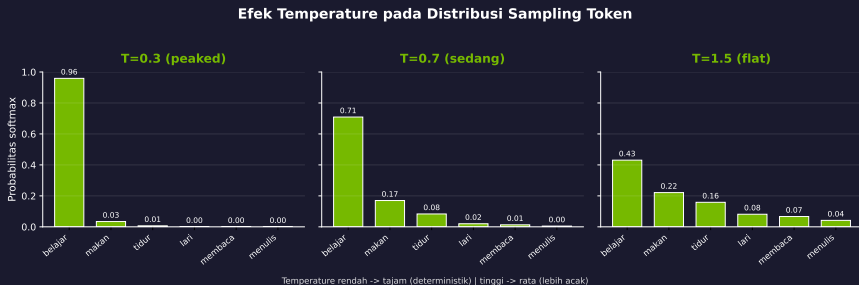
- ▶ **LayerNorm** → merapikan angka biar stabil
- ▶ **Multi-head attention** → tukar konteks antar-kata
- ▶ **Feed-forward** → mengolah tiap kata lebih dalam
- ▶ **Residual** → jalan pintas agar info awal tidak hilang

Tumpuk block ini berkali-kali → makin dalam, makin paham.



Langkah 5 — Menulis Jawaban Token demi Token

Generasi (autoregressive) & cara mengatur gayanya



- ▶ Model menebak **1 token**, lalu token itu jadi input untuk menebak token berikutnya
- ▶ Ulangi terus sampai jawaban selesai
- ▶ **temperature**: rendah = fokus/aman; tinggi = kreatif/lebih acak
- ▶ **top_p** & **repetition_penalty**: atur variasi & cegah pengulangan

Act 3

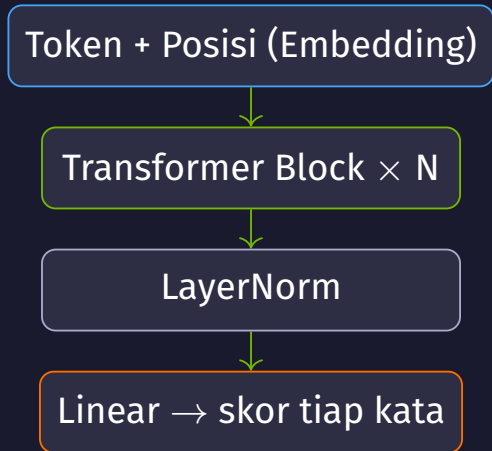
BANGUN: Transformer dari Nol

Notebook 00 — PyTorch saja, tanpa library ajaib

Membangun TinyGPT dari Nol

Menyusun sendiri tiap potongan yang baru dipelajari

- ▶ Hanya pakai **PyTorch** — tanpa library `transformers`
- ▶ **1 huruf = 1 token** (tokenizer paling sederhana)
- ▶ Susun bertahap: **self-attention** → **multi-head** → **transformer block**
- ▶ **Causal mask**: tiap token hanya boleh melihat ke kiri (masa lalu), tidak boleh mengintip masa depan
- ▶ Rakit jadi **TinyGPT** $\approx$ 0,8 juta parameter



TinyGPT $\approx$ 0,8M parameter

Self-Attention dalam Beberapa Baris

Ini adalah “diskusi kelompok” tadi — sebagai bobot, bukan kode

	Ibu	kota	Indonesia	adalah
Ibu	0.90	×	×	×
kota	0.35	0.65	×	×
Indonesia	0.20	0.30	0.55	×
adalah	0.10	0.15	0.80	0.30

baris = kata yang bertanya · kolom = kata yang dilihat · hijau pekat =
bobot besar

- ▶ Tiap kata membuat **Query**; tiap kata punya **Key** + **Value**
- ▶ **Skor** = $Q \cdot K^T \div \sqrt{\text{dim}} \rightarrow \text{softmax} \rightarrow$ **bobot** (tiap baris)
- ▶ **Causal mask** (×) menutup kata di kanan, supaya model menebak **kiri-ke-kanan**

Contoh: “adalah” paling memperhatikan “Indonesia”.

Ini adalah “diskusi kelompok” dari Act 2 — sekarang sebagai bobot, bukan kode.

Latih TinyGPT: dari Acak ke Kalimat

Korpus Indonesia kecil → model bisa melanjutkan kalimat

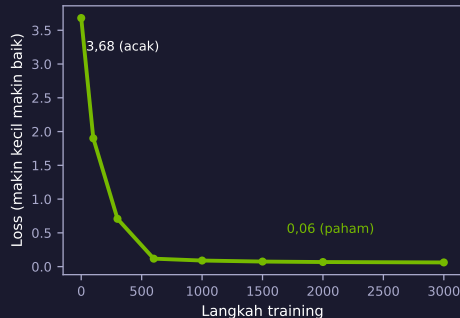
Sebelum latih (step 0)

kMKhcrvwf qzx
jpl wggh tnz

3000 langkah

Sesudah latih (step 3000)

saya suka belajar
jaringan saraf tiruan



Tugasnya cuma: tebak token berikutnya. Loss turun = model makin paham.

TinyGPT ~0,81 juta parameter — prinsip sama dengan GPT raksasa, beda skala.

Act 4

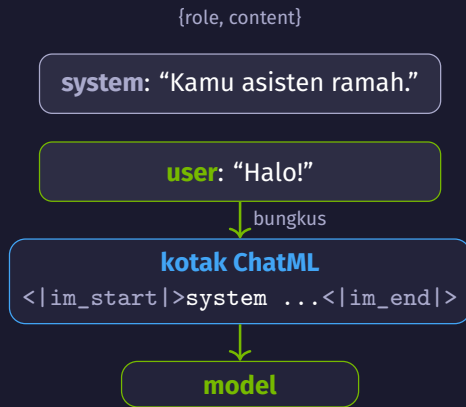
PAKAI: Model Instruct

Notebook 01 — Qwen2.5-3B-Instruct di Colab T4

Chat Template & System Prompt

Cara yang benar untuk “berbicara” dengan model instruct

- ▶ Model chat dilatih dengan **format khusus (ChatML)**
- ▶ `apply_chat_template` menyusun peran *system/user* **persis** seperti saat training
- ▶ **system prompt** = mengatur “kepribadian” & aturan model
- ▶ Jangan rangkai string “User: ...” manual



System prompt = “brief” untuk model; chat template = amplop resmi yang membungkusnya.

Mengatur Gaya Jawaban

Parameter generasi yang sering dipakai

Parameter	Efek
temperature	Rendah = fokus/aman; tinggi = kreatif/acak
top_p	Hanya pilih dari kumpulan kata berprobabilitas terbatas
repetition_penalty	Mengurangi jawaban yang mengulang-ulang
max_new_tokens	Batas panjang jawaban yang dihasilkan

Untuk fakta: temperature rendah. Untuk ide kreatif: naikan temperature & top_p.

Prompt Engineering: Zero-shot & Few-shot

Mengarahkan model lewat cara bertanya

Zero-shot

Teks: ``Kopinya enak
sekali.''

Label?

*(langsung tanya,
tanpa contoh)*

cuma pertanyaan

Few-shot

Teks: ``Filmnya bagus!''

→ **Positif**

Teks: ``Pelayanan lambat.''

→ **Negatif**

Teks: ``Kopinya enak sekali.''

→ ?

2 contoh dulu, lalu item baru

Few-shot = kasih contoh di prompt; model meniru pola tanpa training ulang (*in-context learning*).

Klasifikasi Teks: Head Terlatih vs Belum

Kenapa skor bisa $\sim 0,5$ (sekadar tebakan acak)

- ▶ **bert-tiny**: bagian klasifikasi **belum** dilatih
→ label LABEL_0/1, skor $\sim 0,5$
- ▶ **distilbert-sst2**: sudah dilatih (*fine-tuned*)
→ POSITIVE/NEGATIVE yang benar
- ▶ **bart-mnli** (zero-shot): kita beri *label sendiri*, tanpa melatih

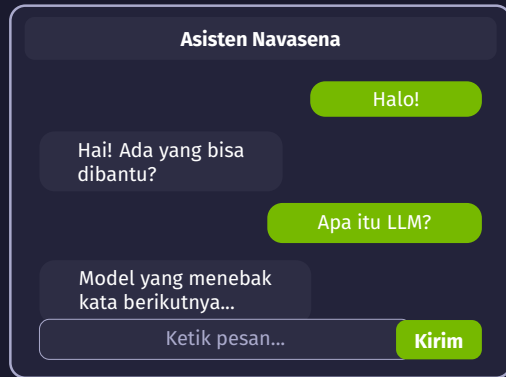
Model	Label	Skor
bert-tiny	LABEL_1	0,51
distilbert-sst2	POSITIVE	0,99
bart-mnli (zero-shot)	"ulasan bagus"	0,94

Bagian klasifikasi belum dilatih → skor $\sim 0,5$ (acak). Untuk hasil nyata: pakai model *fine-tuned* atau zero-shot.

Chatbot dengan Gradio

Antarmuka chat siap pakai, tanpa bikin UI dari nol

- ▶ Bungkus model jadi fungsi `reply(pesan, riwayat)`
- ▶ Gradio **ChatInterface** bikin UI chat instan yang bisa di-*share*
- ▶ `history` (riwayat) disediakan otomatis
- ▶ Satu tautan publik → orang lain bisa langsung coba



Act 5

PRODUKSIKAN: 4-bit & Serving

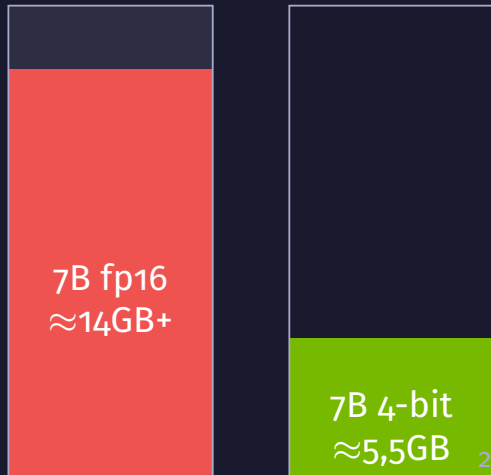
Notebook 02 — Mistral-7B muat di T4 16GB

Kenapa 7B fp16 “Meledak” di T4?

Hitungan kasar memori GPU

- ▶ Tesla T4 punya **~16GB** memori (gratis di Colab)
- ▶ **fp16** = 2 byte per parameter
- ▶ 7 miliar parameter $\times$ 2 byte $\approx$ **14GB** hanya untuk bobotnya
- ▶ Tambah data kerja lain $\rightarrow$ **OOM** (memori habis)
- ▶ Solusi: **kecilkan** bobot lewat **4-bit quantization**

T4: 16GB



4-bit NF4 Quantization

Satu bobot, dari 16-bit ke 4-bit — dan jebakan T4



- ▶ **NF4** menyimpan bobot dalam **4 bit** → model 7B muat di T4
- ▶ Hasil: 7B turun ke **~5,5GB** memori

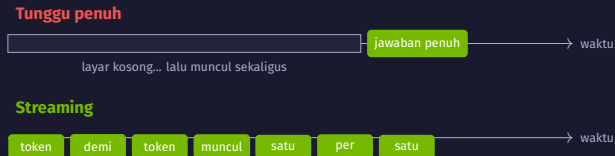
Jebakan T4

`compute_dtype` **WAJIB** `float16`
`bfloat16` crash di T4 (compute 7.5 < 8.0)

Bobot mengecil 4× — tapi di T4, `compute_dtype` harus `float16`.

Profiling, Batching & Streaming

Trik agar layanan model hemat & terasa cepat



- ▶ **Batching** = proses banyak prompt sekaligus
 - ▶ decoder-only → **padding kiri**
- ▶ **Streaming** = tampilkan token saat diproduksi
 - ▶ UX lebih enak

Streaming tidak mempercepat model — ia membuat *pengalaman* terasa jauh lebih cepat.

Act 6

ADAPT: Fine-Tuning LoRA/QLoRA

Notebook 03 — ajari model hal baru tanpa GPU mahal

Masalah: Melatih Ulang Model Raksasa Itu Mahal

Miliaran “kenop” — terlalu berat untuk diubah semua

- ▶ Model raksasa punya **miliaran “kenop”** (angka bobot) yang menentukan perilakunya
- ▶ Untuk mengajarnya hal baru, kita ingin *menyetel* kenop-kenop itu
- ▶ Tapi menyetel **semua** kenop butuh GPU sangat mahal & waktu sangat lama
- ▶ Pertanyaannya: bisakah kita ajari hal baru **tanpa** menyentuh semuanya?

Jawabannya: LoRA.

Model raksasa
miliaran kenop
(mahal diubah semua)

LoRA — Tempel Sticky-Note, Jangan Tulis Ulang Buku

Bekukan model raksasa, latih catatan kecil saja

LoRA = *Low-Rank Adaptation*. Idennya:

- ▶ **Bekukan** model raksasa — jangan sentuh kenopnya sama sekali
- ▶ Tambahkan **matriks kecil** (“catatan tepi”) yang *dilatih*
- ▶ **Analogi**: daripada menulis ulang seluruh buku teks, cukup tempel **sticky-note** kecil berisi koreksi/tambahan
- ▶ Yang dilatih hanya $\sim 1\%$ dari seluruh parameter



Latih hanya sticky-note kecilnya → cepat, murah, mudah disimpan & dibagi.

QLoRA — Buku Edisi Saku 4-bit + Sticky-Note

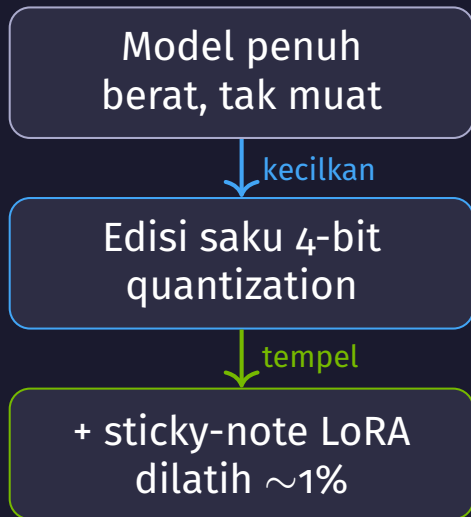
Gabungkan 4-bit dengan LoRA agar muat di T4

QLoRA = **Quantized** LoRA. Dua langkah:

1. **Kecilkan dulu** model ke **4-bit** (quantization) agar muat di GPU kecil
2. **Baru tempel** sticky-note **LoRA** di atasnya

Analogi: buku tebal diubah jadi **edisi saku 4-bit** (ringan dibawa), lalu ditemplei **sticky-note** LoRA.

- ▶ Model 4-bit tetap dibekukan; hanya sticky-note LoRA yang dilatih
- ▶ Inilah kenapa fine-tuning bisa jalan di **T4 gratis** dalam ~ 2 menit



Anatomi QLoRA: Apa yang Dilatih

Notebook 03 — ajari Qwen2.5-1.5B fakta-fakta Navasena



- ▶ **base** dibekukan + di-quantize **4-bit**
- ▶ Cuma **adapter** kecil yang dilatih (sticky-note tadi)
- ▶ Hasilnya disimpan **terpisah** (beberapa MB), bisa ditempel ulang

Qwen2.5-1.5B, 18 contoh, ~2 menit di T4 — **18,4 juta** dari **1,56 miliar** parameter.

Act 7

UKUR: Evaluasi

Notebook 04 — model mana yang benar-benar lebih baik?

Metrik Otomatis

Mengukur kualitas jawaban dengan angka

Metrik	Mengukur apa
ROUGE	Tumpang-tindih kata dengan jawaban acuan (untuk ringkasan)
BLEU	Kecocokan frasa (sering untuk terjemahan)
BERTScore	Kemiripan <i>makna</i> , bukan kata yang persis sama
Perplexity	Seberapa “terkejut” model — makin rendah makin lancar

ROUGE/BLEU = cocok-kata; BERTScore = cocok-makna; Perplexity = kelancaran model.

Apakah Bedanya Nyata?

Skor lebih tinggi belum tentu benar-benar lebih baik

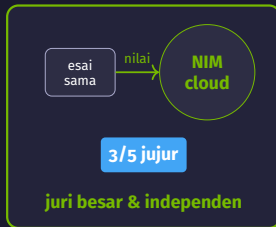
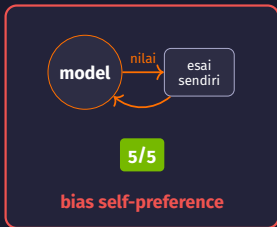
- ▶ Model B skornya sedikit lebih tinggi — tapi apa itu cuma **kebetulan**?
- ▶ Kita uji dengan `p-value` (paired t-test)
- ▶ $p < 0,05 \rightarrow$ beda **nyata** (kecil kemungkinan kebetulan)
- ▶ $p > 0,05 \rightarrow$ **belum** bisa disimpulkan lebih baik

Hasil	Arti
$p = 0,12$	belum nyata
$p = 0,01$	nyata

Pelajaran: metrik boleh memihak model yang lebih besar, tapi $p > 0,05$ berarti bedanya belum meyakinkan.

LLM-as-Judge: Lokal & Cloud

Pakai LLM lain sebagai “juri” untuk menilai jawaban



- ▶ **LLM-as-judge** = pakai LLM lain untuk menilai (*reference-free*, fleksibel)
- ▶ Model yang menilai output **keluarganya sendiri** cenderung memihak
- ▶ Judge cloud besar yang **independen** lebih terpercaya

Lokal: Qwen3B · Cloud: NVIDIA NIM (build.nvidia.com), OpenAI-compatible.

Act 8

Ringkasan & Lanjutan

Lima notebook, satu perjalanan

Ringkasan: 5 Notebook Modul 04

#	Tahap	Model	Yang dipelajari
00	BANGUN	TinyGPT (~0,8M)	Attention, causal mask, multi-head, block — rakit dari nol
01	PAKAI	Qwen2.5-3B-Instruct	Chat template, parameter generasi, prompt eng., klasifikasi
02	PRODUKSIKAN	Mistral-7B-Instruct	4-bit NF4, profiling, batching, streaming
03	ADAPT	Qwen2.5-1.5B	QLoRA: 4-bit + sticky-note, latih ~1%, ~2 menit
04	UKUR	Qwen3B + NIM cloud	ROUGE/BLEU/BERTScore, p-value, LLM-as-judge

Semua gratis di Colab **Tesla T4 16GB**, semua model **non-gated**.

Setup T4 & Lanjut ke Module 05

Aturan main agar tidak crash — dan kenapa kita butuh RAG

Aturan aman di T4:

- ▶ Pin transformers>=4.44,<5
- ▶ 4-bit compute_dtype=float16 (**bf16 crash**)
- ▶ apply_chat_template(..., return_dict=True) lalu generate(**inputs)
- ▶ Cek GPU dengan !nvidia-smi

```
1 !pip install "transformers  
    >=4.44,<5" \  
2             bitsandbytes  
               accelerate  
               peft  
3 !nvidia-smi
```

LLM bisa **mengarang** (halusinasi) & punya **batas pengetahuan** → **Module 05 (RAG)** memberi LLM “buku contekan”.

Terima Kasih!

Pertanyaan? Diskusi?

LLM Fundamentals · Modul 04

Bangun → Pakai → Produksikan → Adapt → Ukur
Navasena Gen-ML Course